CHAMACONNECT

Product Requirements Document

& Agent Execution Specification

Reimagining Digital Chamas for the Future

Version	v1.0 — Hackathon Submission
Date	April 2026
Platform	chamaconnect.io / chamaconnect.co.ke
Organizer	Muiaa Labs × Salamander Community
Target Agents	Cursor, Claude Code, Codex, Google AI Studio
Stack	Next.js 14, TypeScript, Prisma, Base L2, Morpho
Deadline	Monday 11:59 PM EAT

For use by any AI coding agent. All prompts are self-contained.

0. How to Use This Document

This PRD is structured to be consumed directly by AI coding agents (Cursor, Claude Code, Codex, Google AI Studio, Windsurf, etc.). Each feature section includes:

- A plain-language description of what to build
- Precise file paths and component names relative to the existing Next.js project root
- Database schema additions (Prisma)
- API route specifications
- Frontend component specifications
- Integration details (external APIs, SDKs, smart contracts)
- Acceptance criteria — exactly what must be true for the feature to be complete
- Agent prompt — a ready-to-paste prompt the agent can execute directly

> □ *All features extend the existing ChamaConnect codebase. Do not create a new project. Identify the current project structure first before making changes.*
>
> □ *Never commit private keys, RPC URLs with API keys, or wallet mnemonics. Use environment variables exclusively. Add all secrets to .env.local and .env.example (with placeholder values).*

1. Executive Summary

ChamaConnect is a Next.js-based platform serving Kenyan table banking groups (chamas). The current platform provides basic group and member management UI with no live data, no blockchain functionality, no AI features, and no case management — despite marketing all three on the homepage.

This PRD specifies six major feature modules to be built within a single hackathon sprint:

Feature	Priority	Complexity	Module
Blockchain Infrastructure	P0 — Critical	High	Module A
Multi-Sig Wallet Governance	P0 — Critical	High	Module A
DeFi Yield via Morpho	P0 — Critical	High	Module A
ICDMS Case Management	P0 — Critical	Medium	Module B
AI Financial Advisor + CopilotKit	P0 — Critical	Medium	Module C
Smart Dashboard + Demo Mode	P0 — Critical	Low	Module D
Automated Penalty Engine	P1 — High	Low	Module E
M-Pesa / Nobuk / PayHero Integration	P1 — High	Medium	Module F
On-chain Credit Scoring (future)	P2 — Future	Very High	Module G

2. Architecture Overview

2.1 Technology Stack

Frontend	Next.js 14 (App Router), TypeScript, Tailwind CSS, shadcn/ui
Backend	Next.js API routes, Prisma ORM, PostgreSQL (or existing DB)
AI Layer	Anthropic Claude API (claude-sonnet-4-20250514), CopilotKit for agentic actions
Blockchain	Base L2 (EVM-compatible, low fees), Ethers.js v6, Hardhat for contract dev
DeFi	Morpho Protocol (Base deployment), ERC-4337 Account Abstraction for gasless UX
Payments	Safaricom Daraja API (M-Pesa STK Push + C2B), Nobuk API, PayHero webhook
Notifications	Africa's Talking SMS, or existing notification provider in project

2.2 Guiding Principles

- Blockchain is backend infrastructure — users never see wallet addresses, gas fees, or transaction hashes in the primary UI
- All blockchain interactions are abstracted behind familiar Kenyan financial metaphors: 'contribute', 'withdraw', 'pool funds'
- Every new API route is protected by the existing auth middleware — check the project for the auth pattern and replicate it
- Every database change is additive — no existing tables or columns are dropped or renamed
- Demo mode is always available — seed data populates all empty states so judges see a working product

---

MODULE A — Blockchain Infrastructure & DeFi

Priority: P0 · Complexity: High · Files: /contracts, /lib/blockchain, /app/api/blockchain

A.1 Overview & UX Philosophy

ChamaConnect's homepage promises blockchain but delivers none. This module implements real blockchain features while keeping the UX completely non-technical. The goal: a chama treasurer can click 'Pool Funds' and funds move on-chain without ever knowing what a wallet or gas fee is.

Key UX rules for all blockchain features:

- Never show wallet addresses in the primary UI — show member names and KES amounts
- Never mention 'gas', 'wei', 'blockchain', 'smart contract', or 'DeFi' in user-facing copy
- Replace with: 'Secured', 'Verified', 'Pooled', 'Earning', 'Protected by ChamaConnect'
- All blockchain operations are gasless for users via ERC-4337 paymaster or relayer
- Show a simple progress indicator ('Securing your contribution... Done ✓') instead of tx hash

A.2 Smart Contract Suite

A.2.1 ChamaVault.sol — Multi-Signature Group Wallet

Deploy one ChamaVault contract per chama group. Replaces the single-treasurer model.

- Implements M-of-N multi-signature: configurable threshold (e.g. 2-of-3 committee members required to approve withdrawals above a threshold)
- Below-threshold transactions (e.g. under Ksh 5,000) require only 1 signer (individual member contributions)
- Emits events for every action: ContributionReceived, WithdrawalProposed, WithdrawalApproved, WithdrawalExecuted
- Immutable audit log: all events indexed and queryable
- Owner roles: ADMIN (full access), TREASURER (propose withdrawals), MEMBER (contribute only)

Contract Interface (Solidity)

```
// SPDX-License-Identifier: MIT
// contracts/ChamaVault.sol
contract ChamaVault {
  mapping(address => bool) public signers;
  uint256 public threshold;      // e.g. 2 of 3
  uint256 public singleSignLimit; // Ksh equivalent in stablecoin


  function contribute(uint256 amount) external;
  function proposeWithdrawal(address to, uint256 amount, string calldata reason)
external returns (uint256 proposalId);
  function approveWithdrawal(uint256 proposalId) external;
  function executeWithdrawal(uint256 proposalId) external;
  function getBalance() external view returns (uint256);
  function getProposal(uint256 id) external view returns (Proposal memory);
}
```

A.2.2 ChamaFactory.sol — Registry Contract

A singleton factory that deploys and indexes all ChamaVault instances. Enables on-chain discovery of all chamas.

- createChama(string name, address[] signers, uint256 threshold) → deploys ChamaVault

- Maps chamaId → vault address
- Emits ChamaCreated(chamaId, vaultAddress, name) event

A.3 DeFi Yield via Morpho Protocol

A.3.1 What This Does

When a chama's pooled balance exceeds a configurable idle threshold (e.g. Ksh 50,000), idle funds are automatically deployed into a Morpho vault on Base L2 to earn yield. From the member's perspective, they see 'Your chama funds are earning 8.2% APY' — no DeFi knowledge required.

- Uses USDC as the stablecoin (pegged to USD, liquid, available on Base)
- KES amounts are displayed throughout the UI; USD/USDC conversion happens silently using real-time rates
- Morpho vaults on Base provide lending yield (currently 6-12% APY depending on market)
- Withdrawals are instant (Morpho's liquid markets) — funds available within one block

A.3.2 MorphoYieldManager.sol

```
// contracts/MorphoYieldManager.sol
interface IMorpho { ... } // import from @morpho-org/morpho-sol

contract MorphoYieldManager {
  address public immutable morpho;
  address public immutable usdc;
  mapping(address => uint256) public shares; // vault => shares deposited

  function deposit(address vault, uint256 usdcAmount) external;
  function withdraw(address vault, uint256 shares) external returns (uint256
assets);
  function currentYield(address vault) external view returns (uint256 apyBps);
}
```

A.3.3 Idle Fund Detection (Backend)

- Cron job (or Vercel cron) runs every 24h: /app/api/blockchain/yield-check/route.ts
- Queries each chama's vault balance
- If balance > idleThreshold AND no pending withdrawals in next 7 days → auto-deploy to Morpho
- Treasurer gets notification: 'Ksh 85,000 of idle funds are now earning 8.4% APY'

A.4 Collateral Borrowing

Chamas can use their pooled USDC as collateral to borrow additional stablecoins from Morpho lending markets, enabling investments beyond the group's own savings.

- Borrowing UI: 'Access additional funds against your pool' — shows available credit, interest rate, repayment schedule
- Health factor displayed as a simple colored bar (Green = safe, Yellow = caution, Red = at risk)
- Auto-repayment from future contributions if health factor drops below 1.2

- Maximum LTV: 75% of collateral value — hardcoded in contract

> ⚠ *Collateral borrowing involves liquidation risk. Display a plain-language warning in the UI: 'If your group misses contributions and fund value drops, ChamaConnect may automatically repay part of your borrowed funds using your pool.'*

A.5 Future Module — On-Chain Credit Scoring (Module G — Do Not Build Now)

> ⚠ *This is noted here for design awareness only. Do not implement in the hackathon sprint. Future work: Each chama accumulates a ChamaScore NFT updated monthly based on: repayment rate, contribution consistency, dispute resolution history, and loan default rate. A higher ChamaScore enables undercollateralized borrowing at preferential rates through partner DeFi protocols.*

A.6 Blockchain Environment Variables

```
# .env.local additions for Module A
NEXT_PUBLIC_CHAIN_ID=8453             # Base Mainnet (or 84532 for Base Sepolia testnet)
NEXT_PUBLIC_RPC_URL=https://...        # Alchemy/Infura Base RPC
CHAMA_FACTORY_ADDRESS=0x...            # Deployed ChamaFactory address
MORPHO_YIELD_MANAGER_ADDRESS=0x...     # Deployed MorphoYieldManager address
RELAYER_PRIVATE_KEY=0x...              # Backend relayer wallet (pays gas on behalf of users)
USDC_ADDRESS=0x833589fCD6eDb6E08f4c7C32D4f71b54bdA02913 # USDC on Base
```

A.7 File Structure for Module A

```
/contracts/
  ChamaVault.sol
  ChamaFactory.sol
  MorphoYieldManager.sol
  hardhat.config.ts
/lib/blockchain/
  client.ts          # ethers.js provider + signer setup
  vault.ts           # ChamaVault read/write helpers
  factory.ts         # ChamaFactory helpers
  morpho.ts          # Morpho interaction helpers
  types.ts           # TypeScript types for all on-chain data
/app/api/blockchain/
  deploy-vault/route.ts     # POST: create new ChamaVault on chain
  contribute/route.ts       # POST: relay user contribution to vault
  propose-withdrawal/route.ts
  approve-withdrawal/route.ts
  yield-status/route.ts     # GET: current APY + deposited amount
  yield-check/route.ts      # CRON: auto-deploy idle funds
```

A.8 Agent Prompt — Module A

AGENT PROMPT — Module A: Blockchain Infrastructure

You are working on the ChamaConnect platform (chamaconnect.io). This is a Next.js 14 + TypeScript project. First, run `ls` and `cat package.json` to understand the current project structure. Then: (1) Install dependencies: ethers@6, hardhat, @openzeppelin/contracts, @morpho-org/morpho-sol. (2) Create /contracts/ directory with ChamaVault.sol, ChamaFactory.sol, MorphoYieldManager.sol as specified in the PRD. (3) Create /lib/blockchain/ helpers with full TypeScript types. (4) Create all API routes under /app/api/blockchain/. (5) Add all required env vars to .env.example with placeholder values. (6) Ensure all blockchain calls from the frontend show simple progress states, never raw transaction data. Do not modify existing files — only add new ones unless you must extend an existing type or schema.

MODULE B — ICDMS: Integrated Case & Dispute Management

Priority: P0 · Complexity: Medium · Files: /app/(dashboard)/cases, /app/api/cases, prisma/schema.prisma

B.1 Overview

The ICDMS is the core requirement explicitly named in the hackathon brief. It is a structured case management system for chama disputes, financial exceptions, and member issues. Every action is logged immutably (on-chain event via ChamaVault's audit log, and mirrored in the database for fast querying).

B.2 Database Schema (Prisma additions)

```
// prisma/schema.prisma — ADD these models

enum CaseStatus {
  OPEN UNDER_REVIEW RESOLVED DISMISSED ESCALATED
}

enum CaseType {
  MISSED_CONTRIBUTION LATE_PAYMENT LOAN_DISPUTE
  MEMBER_COMPLAINT FRAUD_ALLEGATION PENALTY_APPEAL OTHER
}

model Case {
  id            String     @id @default(cuid())
  chamaId       String
  reportedById  String     // userId
  againstId     String?    // userId (optional, can be against group)
  assignedToId  String?    // reviewer userId
  type          CaseType
  status        CaseStatus @default(OPEN)
  title         String
  description   String     @db.Text
  amountKes     Float?     // if financial
  aiSuggestion  String?    @db.Text // Claude's resolution suggestion
  onChainTxHash String?    // reference to ChamaVault audit log
  resolvedAt    DateTime?
  createdAt     DateTime   @default(now())
  updatedAt     DateTime   @updatedAt
  chama         Chama      @relation(fields: [chamaId], references: [id])
  events        CaseEvent[]
  attachments   CaseAttachment[]
}

model CaseEvent {
```

```
  id        String   @id @default(cuid())
  caseId    String
  actorId   String
  action    String   // 'OPENED', 'STATUS_CHANGED', 'COMMENT', 'ASSIGNED',
'RESOLVED'
  note      String?  @db.Text
  metadata  Json?
  createdAt DateTime @default(now())
  case      Case     @relation(fields: [caseId], references: [id])
}

model CaseAttachment {
  id        String @id @default(cuid())
  caseId    String
  url       String // uploaded file URL
  name      String
  case      Case   @relation(fields: [caseId], references: [id])
}
```

B.3 API Routes

GET /api/cases	List all cases for current user's chamas. Filter by status, type, chamaId.
POST /api/cases	Create a new case. Body: { chamaId, type, title, description, amountKes?, againstId? }
GET /api/cases/[id]	Get single case with all events and attachments.
PATCH /api/cases/[id]	Update status, assignee, or add a resolution note.
POST /api/cases/[id]/events	Add a new event/comment to the case timeline.
POST /api/cases/[id]/ai-suggest	Trigger Claude API to analyze the case and return a resolution suggestion.
GET /api/cases/stats	Dashboard stats: open count, avg resolution time, by-type breakdown.

B.4 AI Resolution Suggestion (Claude API)

When a reviewer clicks 'Get AI Suggestion', the system sends the case details to Claude and returns a structured recommendation:

```
// /app/api/cases/[id]/ai-suggest/route.ts
const prompt = `
You are a Kenyan chama arbitration advisor. Analyze this case:

Chama: ${chama.name}
Case type: ${caseType}
Description: ${caseDescription}
Group rules: ${groupRules}
Member history: ${memberHistory}

Provide:
1. Recommended resolution (1-2 sentences)
2. Precedent from group rules (cite the rule if applicable)
3. Fairness score (1-10, 10 = fully fair to both parties)
4. Action items for the treasurer (max 3 bullet points)

Respond in the language the case was written in (English or Swahili).`
```

B.5 Frontend Components

- /app/(dashboard)/cases/page.tsx — Case list with filters, status badges, priority flags
- /app/(dashboard)/cases/[id]/page.tsx — Case detail: timeline, AI suggestion panel, attachment viewer
- /app/(dashboard)/cases/new/page.tsx — Case creation form
- /components/cases/CaseCard.tsx — Summary card used in list and dashboard
- /components/cases/CaseTimeline.tsx — Vertical timeline of CaseEvent entries
- /components/cases/AISuggestionPanel.tsx — Displays Claude's suggestion with confidence indicator
- /components/cases/StatusBadge.tsx — Colored badge: OPEN=amber, RESOLVED=green, ESCALATED=red

B.6 Dashboard Integration

Add a 'Cases' section to the main dashboard showing: open case count, cases assigned to me, and a 'Flag a Case' quick-action button that opens a modal with the new case form.

B.7 Acceptance Criteria

- A member can create a case of any CaseType from the dashboard in under 3 clicks
- A reviewer can change case status and add timeline events
- AI suggestion appears within 5 seconds of clicking 'Get AI Suggestion'
- All case events are immutable once written (no DELETE routes for CaseEvent)
- Case stats appear on the main dashboard
- Demo mode seeds: 3 open cases, 2 resolved, 1 escalated across 2 chama groups

B.8 Agent Prompt — Module B

> *AGENT PROMPT — Module B: ICDMS. Add the Prisma models from Section B.2 to the existing prisma/schema.prisma. Run `npx prisma migrate dev --name add_icdms_cases`. Create all API routes under /app/api/cases/. Create all frontend pages and components as listed in B.5. The case creation form must be accessible from the main dashboard. Integrate the Claude API for AI suggestions using the prompt template in B.4. Add demo seed data (6 cases) to /prisma/seed.ts. All routes must use the existing auth middleware pattern.*

MODULE C — Chama AI Financial Advisor + CopilotKit

Priority: P0 · Files: /app/(dashboard)/advisor, /app/api/copilotkit, /components/advisor

C.1 Overview

An embedded AI assistant that answers chama-specific financial questions using real group data. Powered by Claude API + CopilotKit, the advisor can take direct actions inside the platform (not just suggest them).

> □ *CopilotKit (https://github.com/copilotkit/copilotkit) enables the AI to perform actions: creating cases, recording contributions, sending notifications. Install: npm install @copilotkit/react-core @copilotkit/react-ui @copilotkit/backend*

C.2 Advisor Capabilities

Answerable Questions (read-only)

- 'How much can I borrow based on my contribution history?' → calculates based on group loan policy and member history
- 'What is our group's net position this month?' → queries live financial data
- 'Who has missed contributions in the last 3 months?'
- 'What is our projected fund balance in 6 months if growth continues?'
- 'How much yield have we earned from DeFi this month?'
- 'Niambie jinsi ya kuwasiliana na mwanachama aliyechelewa.' (Swahili: 'Tell me how to contact a late member.')

Actions via CopilotKit (agentic)

- Create a new ICDMS case for a member → calls POST /api/cases
- Send a payment reminder to overdue members → calls notification API
- Record a manual contribution → calls POST /api/contributions
- Generate and download a monthly report → calls report generation endpoint
- Draft a penalty notice letter → generates document, offers download

C.3 CopilotKit Integration

```
// /app/api/copilotkit/route.ts
import { CopilotBackend, AnthropicAdapter } from '@copilotkit/backend';

export async function POST(req: Request) {
  const copilotKit = new CopilotBackend({
    actions: [
      {
        name: 'createCase',
        description: 'Create a new ICDMS case for a member issue',
        parameters: [
          { name: 'type', type: 'string', description: 'Case type' },
          { name: 'memberId', type: 'string' },
```

```
          { name: 'description', type: 'string' },
        ],
        handler: async ({ type, memberId, description }) => {
          // call /api/cases internally
        }
      },
      // ... more actions
    ]
  });
  return copilotKit.response(req, new AnthropicAdapter({ model: 'claude-sonnet-4-20250514' }));
}
```

C.4 System Prompt

```
You are the ChamaConnect Financial Advisor, an AI assistant embedded in a Kenyan table banking platform. You help chama members and treasurers manage their group finances with clarity and confidence.

Current user: ${userName} (${userRole})
Chama: ${chamaName} | Members: ${memberCount} | Balance: Ksh ${balance}

Rules:
- Always respond in the language used by the user (English or Swahili)
- Refer to amounts in Kenyan Shillings (Ksh) unless asked otherwise
- Never mention 'blockchain', 'DeFi', 'smart contracts', or 'wallets'
- Use 'secured', 'verified', 'pooled', 'earning' instead
- When suggesting an action you can take, ask for confirmation first
- Be conversational and warm — chamas are community institutions
```

C.5 UI Component

- /components/advisor/AdvisorChat.tsx — Floating chat widget (bottom-right), collapsible
- /components/advisor/AdvisorMessage.tsx — Message bubble with action confirmation cards
- /components/advisor/ActionConfirmCard.tsx — Shows proposed action with Confirm/Cancel
- Use CopilotKit's <CopilotChat> component as the base, customized with ChamaConnect styling
- Available on every dashboard page via a persistent floating button

C.6 Agent Prompt — Module C

> *AGENT PROMPT — Module C: AI Advisor. Install @copilotkit/react-core @copilotkit/react-ui @copilotkit/backend. Create /app/api/copilotkit/route.ts as specified. Create the floating AdvisorChat component and add it to the root layout so it appears on all dashboard pages. Wire up all 5 CopilotKit actions: createCase, sendReminder, recordContribution, generateReport, draftLetter. Use the system prompt from C.4. All money amounts in the chat must display in Ksh format. Test: ask the advisor 'What is our group balance?' and verify it returns live data.*

MODULE D — Smart Dashboard with Demo Mode

Priority: P0 · Files: /app/(dashboard)/page.tsx, /components/dashboard, /lib/demo-data.ts

D.1 Problem Statement

The current dashboard shows Ksh 0.00 in all three metric cards, empty Group Comparison, and empty My Goals and Recent Transactions panels. This is the first screen judges see. It must show real, engaging data.

D.2 Demo Mode

A toggle in the top-right of the dashboard activates Demo Mode, which injects realistic seed data without touching production records:

- Demo toggle is visually prominent: amber badge 'DEMO MODE ON' in the header
- All data comes from /lib/demo-data.ts — a static file of realistic Kenyan chama data
- Demo data includes: 2 chama groups, 8 members each, 6 months of contribution history, 3 loan records, 2 active goals, 6 ICDMS cases

D.3 Dashboard Sections to Build/Enhance

D.3.1 Metric Cards (top row)

- Total Contributions — sum of all confirmed contributions across all user's chamas
- Income Analysis — total loan interest earned + DeFi yield
- Expense Analysis — total fees, penalties paid
- New card: DeFi Yield Earned — amount earned from Morpho in current month, with trend arrow

D.3.2 Contribution Trend Chart

- Line/area chart: last 6 months of group contributions
- Use recharts or Chart.js (check what's already in package.json)
- Toggle between weekly and monthly view
- Show per-member breakdown on hover

D.3.3 AI Monthly Summary

- Calls /api/dashboard/ai-summary which queries Claude with the month's data
- Returns 2-3 sentences: 'Your chama contributed Ksh 84,000 in April, up 12% from March. 2 members have missed 2+ consecutive contributions. Your DeFi allocation is earning 8.2% APY.'
- Displayed as a highlighted card at the top of the dashboard
- Refreshes monthly (cached in DB with createdAt)

D.3.4 Member Health Scores

- Each member gets a health score 0-100 based on: contribution consistency (50%), loan repayment (30%), case history (20%)
- Displayed as a mini scoreboard on the dashboard — top 3 and bottom 3 members

D.3.5 Quick Actions

- Record Contribution → opens modal with member selector + amount

- Issue Loan → opens loan creation flow
- Flag Case → opens ICDMS new case form (Module B)
- View Reports → links to report export page

D.3.6 Upcoming Calendar

- Next contribution due date with countdown
- Upcoming loan repayment dates
- Scheduled meeting dates (if any)

D.4 Agent Prompt — Module D

> *AGENT PROMPT — Module D: Smart Dashboard. First examine the existing /app/(dashboard)/page.tsx and all existing dashboard components. Then: (1) Create /lib/demo-data.ts with realistic Kenyan chama data (names, amounts in Ksh, Swahili group names). (2) Add a Demo Mode toggle that switches between real data and demo data. (3) Add a contribution trend chart using whatever charting library is already in the project. (4) Add an AI monthly summary card that calls /api/dashboard/ai-summary. (5) Add member health scores. (6) Add quick-action buttons. All existing dashboard UI must be preserved — only add, never remove. Ensure empty states are only shown if Demo Mode is off AND real data is genuinely empty.*

MODULE E — Automated Penalty & Fine Engine

Priority: P1 · Files: /app/api/penalties, /app/(dashboard)/settings/penalties, prisma/schema.prisma

E.1 Overview

A rules engine that automatically calculates fines when members miss contribution deadlines. Integrates directly with ICDMS (auto-creates a case) and the notification system.

E.2 Database Schema

```
model PenaltyRule {
  id             String   @id @default(cuid())
  chamaId        String
  name           String   // 'Late Contribution Fine'
  amountKesPerDay Float   // e.g. 200
  maxDays        Int      // e.g. 5
  gracePeriodDays Int     @default(0)
  autoCreateCase Boolean  @default(true)
  isActive       Boolean  @default(true)
}

model PenaltyRecord {
  id           String   @id @default(cuid())
  chamaId      String
  memberId     String
  ruleId       String
  amountKes    Float
  daysLate     Int
  status       String   @default('PENDING') // PENDING | PAID | WAIVED | DISPUTED
  caseId       String?  // linked ICDMS case if disputed
  createdAt    DateTime @default(now())
}
```

E.3 Engine Logic

```
// /app/api/penalties/calculate/route.ts  (called by CRON daily)
1. Find all chamas with active PenaltyRules
2. For each chama, find members whose contribution_due_date has passed
3. Calculate daysLate = today - due_date - gracePeriod
4. amountKes = min(daysLate, maxDays) * amountKesPerDay
5. Create PenaltyRecord if not already exists for this period
6. If autoCreateCase: POST to /api/cases with type=LATE_PAYMENT
7. Send SMS notification to member via Africa's Talking
```

E.4 Settings UI

- /app/(dashboard)/settings/penalties/page.tsx — Admin-only page
- Create and edit penalty rules with a simple form
- View all pending penalties with Pay / Waive / Dispute actions
- Dispute triggers ICDMS case creation

E.5 Agent Prompt — Module E

> *AGENT PROMPT — Module E: Penalty Engine. Add PenaltyRule and PenaltyRecord models to prisma/schema.prisma. Run migration. Create /app/api/penalties/ routes: GET (list), POST (create rule), PATCH (update status). Create a daily CRON route at /app/api/penalties/calculate/route.ts that runs the engine logic from E.3. Create the settings UI at /app/(dashboard)/settings/penalties/. Integrate with ICDMS: when autoCreateCase is true, call the case creation logic. Add a 'Penalties' section to the My Chamas page showing each member's outstanding fines.*

MODULE F — M-Pesa / Nobuk / PayHero Integration

Priority: P1 · Files: /app/api/payments, /lib/payments, /components/payments

F.1 Overview

Integrates three payment providers for contribution collection and reconciliation. All three are optional — the system degrades gracefully if credentials are absent.

F.2 M-Pesa (Safaricom Daraja API)

STK Push (Lipa Na M-Pesa)

- Member clicks 'Pay Contribution' → enters their Safaricom phone number → STK push sent to their phone
- On M-Pesa confirmation → webhook received at /api/payments/mpesa/callback
- Webhook handler: verify signature, parse amount + phone, match to member, create Contribution record
- If amount mismatch: flag for treasurer review → auto-create ICDMS case type MISSED_CONTRIBUTION

```
// /lib/payments/mpesa.ts
export async function initiateStkPush({ phone, amountKes, chamaId, memberId }) {
  const token = await getAccessToken(); // Daraja OAuth
  return fetch('https://sandbox.safaricom.co.ke/mpesa/stkpush/v1/processrequest', {
    method: 'POST',
    headers: { Authorization: `Bearer ${token}` },
    body: JSON.stringify({
      BusinessShortCode: process.env.MPESA_SHORTCODE,
      Password: generatePassword(),
      PhoneNumber: formatKenyanPhone(phone),
      Amount: Math.ceil(amountKes),
      CallBackURL: `${process.env.NEXT_PUBLIC_URL}/api/payments/mpesa/callback`,
      AccountReference: `CHAMA-${chamaId}`,
    })
  });
}
```

F.3 Nobuk Integration

Nobuk (nobuk.com) is a Kenyan digital banking API for business accounts. Use for group wallet top-ups and disbursements where M-Pesa is insufficient.

- POST /api/payments/nobuk/transfer — initiate transfer from Nobuk account
- GET /api/payments/nobuk/transactions — sync transaction history
- Webhook at /api/payments/nobuk/webhook — receive real-time transaction events

> □ *Nobuk API credentials required. Add NOBUK_API_KEY and NOBUK_ACCOUNT_ID to .env.local. Fall back gracefully (hide UI) if not set.*

F.4 PayHero Integration

PayHero (payhero.co.ke) provides M-Pesa B2C, C2B, and card payments via a unified API — useful for chamas needing a payment page link.

- POST /api/payments/payhero/charge — initiate payment
- Webhook at /api/payments/payhero/webhook — receive payment confirmation
- Generate a shareable payment link per contribution cycle: 'Pay your April contribution →'

> □ *PayHero API key required. Add PAYHERO_API_KEY to .env.local. Feature hidden if not configured.*

F.5 Reconciliation Engine

- Every incoming payment webhook is matched to a PendingContribution by: phone number + amount + chamaId + within 48h window
- Exact match → auto-confirm contribution, update status to PAID
- Amount mismatch (±10%) → flag as PARTIAL, notify treasurer
- No match → create unmatched payment record, notify treasurer to manually assign
- All reconciliation actions logged in a ReconciliationLog table

F.6 Environment Variables

```
MPESA_CONSUMER_KEY=...
MPESA_CONSUMER_SECRET=...
MPESA_SHORTCODE=...
MPESA_PASSKEY=...
NOBUK_API_KEY=...
NOBUK_ACCOUNT_ID=...
PAYHERO_API_KEY=...
NEXT_PUBLIC_URL=https://chamaconnect.io  # for webhook URLs
```

F.7 Agent Prompt — Module F

> *AGENT PROMPT — Module F: Payments. Create /lib/payments/ with mpesa.ts, nobuk.ts, and payhero.ts helper modules. Create API routes: /api/payments/mpesa/initiate, /api/payments/mpesa/callback, /api/payments/nobuk/transfer, /api/payments/nobuk/webhook, /api/payments/payhero/charge, /api/payments/payhero/webhook. Create a ReconciliationLog Prisma model. Implement the reconciliation engine that matches incoming webhooks to PendingContributions. All payment provider integrations must be conditional on environment variables being set — if the key is missing, the feature is hidden in the UI. Add a 'Pay Contribution' button to the member dashboard that triggers M-Pesa STK Push.*

3. Cross-Cutting Concerns

3.1 Notification System

- All notifications sent via: in-app toast + SMS (Africa's Talking) + optional email
- Notification triggers: new case, case resolved, contribution received, penalty applied, loan approved, DeFi yield milestone
- Create a unified /lib/notifications.ts: sendNotification({ userId, type, message, channel[] })

3.2 Authentication & Authorization

- Use the existing auth system — do not replace or modify it
- Roles: SUPER_ADMIN, CHAMA_ADMIN, TREASURER, MEMBER
- Every new API route checks role with: const { user } = await requireAuth(req) and verifies role before proceeding
- Blockchain routes additionally verify the user is a registered signer on the ChamaVault

3.3 Swahili Localisation

- All user-facing strings should support both English and Swahili
- Use next-i18next or a simple translation map in /lib/i18n/sw.ts and /lib/i18n/en.ts
- AI advisor responds in the language the user writes in — this is handled automatically by Claude
- Date formats: DD/MM/YYYY (Kenyan standard)
- Currency: always Ksh with comma-formatted thousands (Ksh 84,000)

3.4 Error Handling Standard

```
// All API routes return this shape:
{ success: true, data: T }          // on success
{ success: false, error: string }    // on error

// HTTP status codes:
200 — OK
201 — Created
400 — Bad Request (validation error)
401 — Unauthenticated
403 — Unauthorized (wrong role)
404 — Not Found
500 — Internal Server Error (log but do not expose details)
```

3.5 Testing

- Each module should have at minimum: 1 Vitest unit test for core logic, 1 API route test using msw or similar
- Seed data in /prisma/seed.ts must be runnable with: npx prisma db seed
- Demo mode toggle in the UI must never affect production data

3.6 GitHub Submission Checklist

- Public repository with MIT license (required by hackathon terms)
- .env.example with all required keys and placeholder values (NO real secrets)
- README.md with: setup instructions, feature list, architecture diagram, demo video link
- All third-party libraries listed in package.json (auto-satisfied by npm install)
- No .env.local or any file with real credentials committed
- Codebase must build without errors: npm run build

---

4. Recommended Implementation Order

For a Monday deadline starting Friday, execute in this sequence:

Day	Module	Goal
Friday	Module D	Fix dashboard: demo mode, charts, AI summary. Judges' first impression must be great.
Friday	Module B	ICDMS core: Prisma schema, API routes, case list + detail pages.
Saturday	Module A	Blockchain: Deploy contracts to Base Sepolia testnet. Wire up vault + factory helpers.
Saturday	Module C	AI Advisor: CopilotKit setup, floating chat widget, 3 working actions.
Sunday	Module E	Penalty engine: rules, CRON, settings UI, ICDMS integration.
Sunday	Module F	M-Pesa STK Push + webhook + reconciliation engine.
Monday	All	Polish, README, env.example, build check, video demo. Submit before 11:59 PM EAT.

---

5. Master Agent Prompt (Paste This First)

Paste this into your coding agent before starting any module. It establishes context and prevents the agent from making destructive changes.

```
—— MASTER CONTEXT PROMPT — CHAMACONNECT HACKATHON ——

You are working on ChamaConnect (chamaconnect.io), a Kenyan table banking
platform for chama groups. This is an existing Next.js 14 + TypeScript project.

BEFORE WRITING ANY CODE:
1. Run: ls -la && cat package.json && cat prisma/schema.prisma
2. Understand the existing file structure, auth pattern, and DB schema
3. Check which UI components and libraries are already installed
4. Never overwrite existing files — add new ones unless explicitly extending
```

```
ABSOLUTE RULES:
- Never commit secrets. All keys go in .env.local (gitignored) + .env.example
- Never drop or rename existing Prisma models or columns
- Never remove existing UI — only add new sections/pages/components
- Blockchain = backend only. UI shows KES amounts, member names, simple statuses
- All money displays formatted as: Ksh 84,000 (comma-separated thousands)
- Support English and Swahili in all user-facing text
- Every new API route must check authentication using the existing auth middleware

HACKATHON CONTEXT:
- Deadline: Monday 11:59 PM EAT
- Judging criteria: ICDMS optimization + feature enhancement
- The judges will log in and immediately see the dashboard
- Demo mode must be on by default so judges see a populated product

Now, tell me what you see in the current project structure before we begin.
─── END MASTER PROMPT ───
```

ChamaConnect PRD & Agent Execution Spec — Muiaa Labs Hackathon 2026
Built with by the hackathon team. Good luck.